Team members: Alex Bagnoni, David Harrington, Jacob Jenkins

Abstract

This study investigates a steady stat two-dimensional temperature distribution within an electronically packed container subjected to internal heat generation and convection cooling. In the z direction, the container is assumed to be infinite, and temperature variation is negligible in this direction. The objective of this project was to write a generalized code which can solve any given m and n values where m corresponds to the number of rows and n is the number of column grid points. This code will solve the steady state 2D internal temperature profile at different convection coefficients. Through a grid independence study, it was observed that an 11x11 grid would produce accurate results while minimizing computational efforts. Increasing the value of the convective coefficient on the y face will decrease the temperature at every node. By using x-centerline, y-centerline, and contour plots, this claim can be confirmed.

Introduction

The thermal management of electronic devices is a critical factor that directly influences their reliability, performance, and operational lifetime. Electronic components generate heat during operation, and if this heat is not adequately dissipated, elevated temperatures can lead to gradual damage of materials, reduced efficiency, or complete device failure. For systems where electronic devices are densely packed inside an enclosure, predicting the internal temperature distribution becomes essential for ensuring safe operating conditions and effective cooling strategies. In this project, the steady-state temperature distribution inside a container filled with heat-generating electronic components is analyzed using a two-dimensional heat conduction model.

The container is cooled externally by a fast-moving fluid maintained at a constant temperature of 25°C. Additionally, the length in the z-direction is assumed to be one. As a result, heat transfer can be simplified to a two-dimensional plane. Due to the different thermal properties of packed material inside the container, it has a different thermal conductivity in the x and y directions.

$$k_x = 7\ \frac{W}{m \cdot K} \qquad k_y = 14\ \frac{W}{m \cdot K}$$

Furthermore, heat is generated uniformly throughout the container at a constant rate of $250{,}000\ \frac{W}{m^3}$. The two surfaces aligned along the x-direction experience convection with a constant heat transfer coefficient of $100\ \frac{W}{m^2 \cdot K}$. The boundary along the y-direction, however, experiences varying convection conditions. In this study, the heat transfer on this surface varied across five values: 10 , 30 , 50 , 70 , and $100\ \frac{W}{m^2 \cdot K}$. By analyzing these different convection coefficients, the influence of external cooling intensity on the internal temperature distribution can be evaluated. These characteristics make the problem well suited for numerical solution techniques.

A generalized computational approach was developed in this project to allow flexibility in the resolution of the numerical model using MATLAB. The domain is discretized using *m* grid points in the x-direction and *n* grid points in the y-direction. By defining the problem in this generalized form, the developed code can solve the temperature field for any chosen grid density,

allowing the user to easily refine the mesh or evaluate grid convergence. The discretized energy balance equations are applied at each internal node, while modified equations incorporating convection boundary conditions are applied along the edges and corners of the domain. This results in a system of linear equations representing the temperature at each grid point.

Through this approach, the project demonstrates the application of numerical methods to predict temperature distributions in materials with internal heat generation, direction dependent conduction, and mixed boundary conditions. The results help illustrate the impact of convective cooling strength on the thermal behavior of electronic systems and provide a framework that can be extended to more complex thermal management problems.

Results and Discussion

To ensure the accuracy and reliability of temperatures in electronic devices, a grid independence study was conducted prior to obtaining the results. To accomplish this, the code was ran using different grid sizes to observe and determine the point at which further grid size increases produced negligible change to the final solution. The maximum temperature was used as a benchmark variable for comparison. The results for the grid independence study can be found below in table (1).

Grid Size (m x n)	Max Temperature (C°)	Percent change from Previous temperature (C°)
4 x 4	76.40	%7.91
6 x 6	83.38	%9.14
9 x 9	88.1892	%5.77
11 x 11	89.83	%1.86

Table 1: Grid independence study results showing maximum domain temperature.

Based on table 1, it was found that the 11 x 11 grid produced accurate results while minimizing computer computation. Graphed results can be seen below in figure 1.

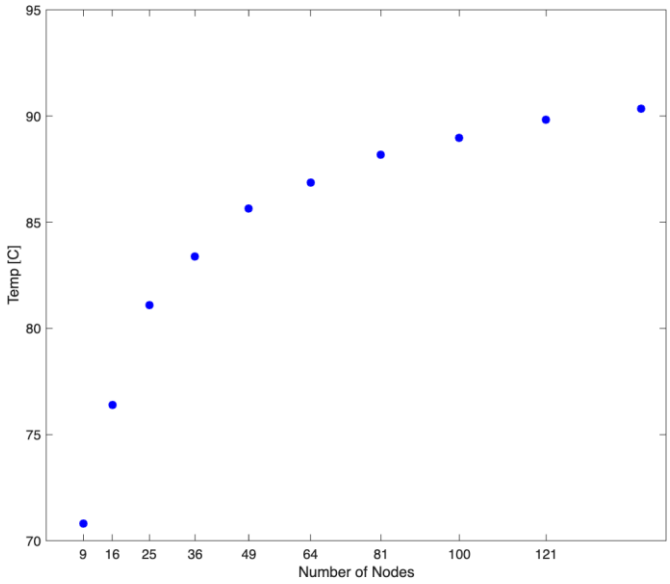


Figure 1. Grid Independence showing Temperature vs Number of Nodes.

To quantify the effect of the convection coefficient on the y face of the internal temperature distribution, temperatures were extracted at five evenly spaced points along the x-centerline and the y-centerline. Different convective coefficients(h) were used, those being (10, 30, 50, 70, 100). The results are presented in the line plots below. As shown in figure 2, the temperature at five nodes is shown along the x-centerline as a function of h. Across all nodes, temperature decreases as h increases from 10 to 100, showing that better convective cooling on the y-face reduces the internal temperatures throughout the domain. At h = 10, node 1 records the lowest temperature of approximately 80.2 Cº while node 5 records the highest temperature of 88.8C. At the maximum convective coefficient, h = 100, these temperature values decreased to 73 Cº and 81.5 Cº. It can be observed that for all 5 node curves, the lines are almost parallel and decrease at a similar rate with increasing h values.

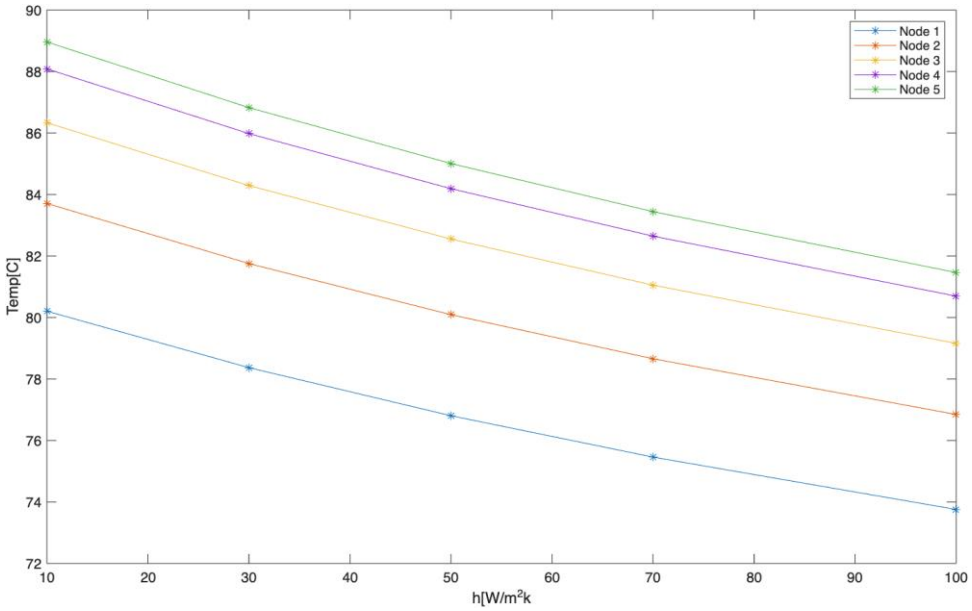


Figure 2. X-Centerline Temperature vs h

The temperature variation at five nodes along the y-centerline as a function of h can be seen in figure 3. In contrast to the x-centerline plots, the y-centerline plot exhibits a non-uniform response to the increasing h value. When h = 10, the temperature at node 1 is approximately 87 degrees while the temperature at node 5 is 89 degrees. This indicates a small temperature spread along the y-centerline when the convective coefficient is weak. However, when h increases to 100, the temperature spread greatly increases. Node 1 temperature drops to 69 degrees while node 5 drops to 81 degrees. The temperature spread increases as the h value increases. This behavior indicates that nodes that are closer to the y-face will be more affected by the convection coefficient, which is caused by its direct cooling.

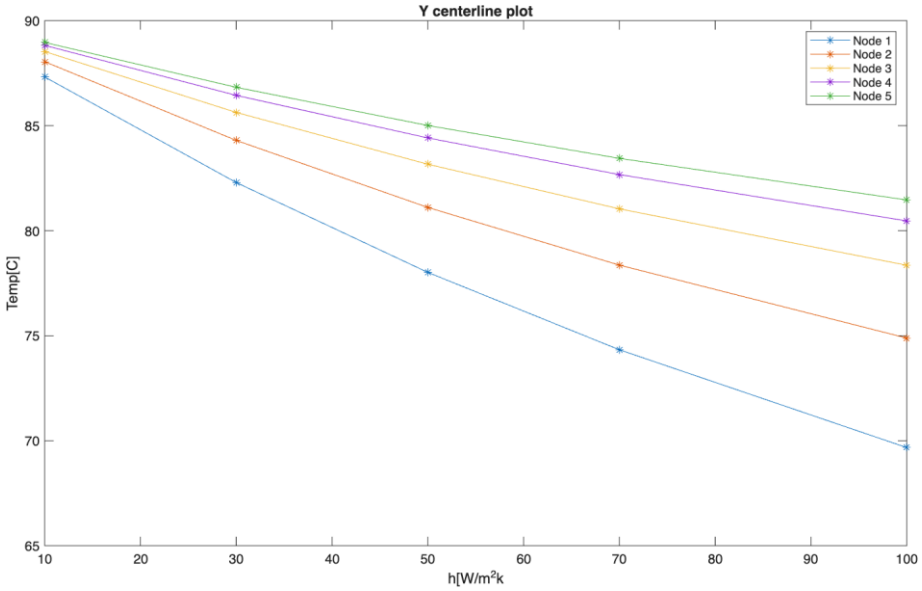


Figure 3: Y-centerline temperature vs h

To develop an effective means of visualizing the temperature distribution inside a m x n size grid, a contour plot was utilized. This allowed us to analyze how the distribution was affected when changing the convective coefficient in the y direction and the conduction coefficient in both directions. In comparing figures 4 and 5 that have the same conduction coefficients, it's noted that an increasing convection coefficient increases heat distribution in the x direction. This effectively means that heat is dissipated faster out the top and bottom of the grid.

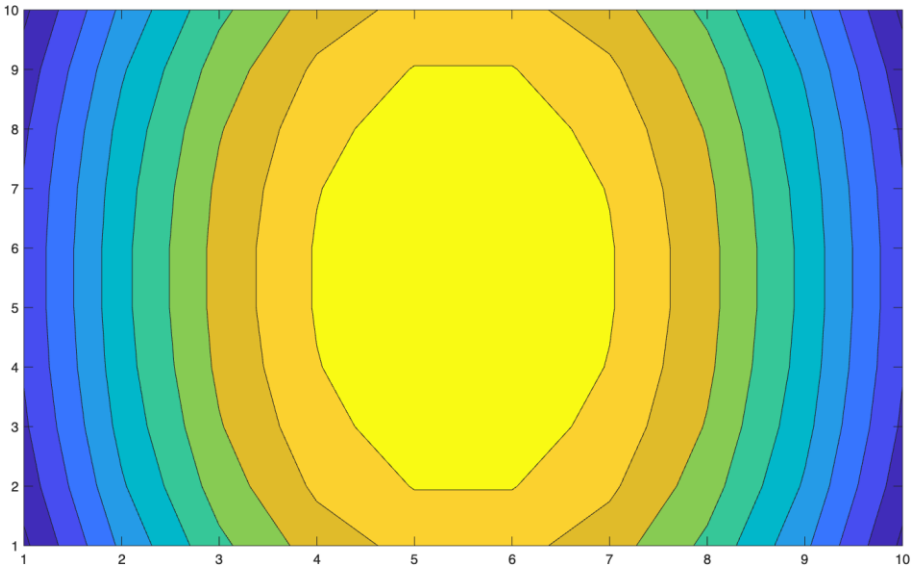


Figure 4: Temperature contours at $h_y = 10\ \frac{W}{m^2 \cdot K}$, $k_x = 7\ \frac{W}{m \cdot K}$, $k_y = 14\ \frac{W}{m \cdot K}$

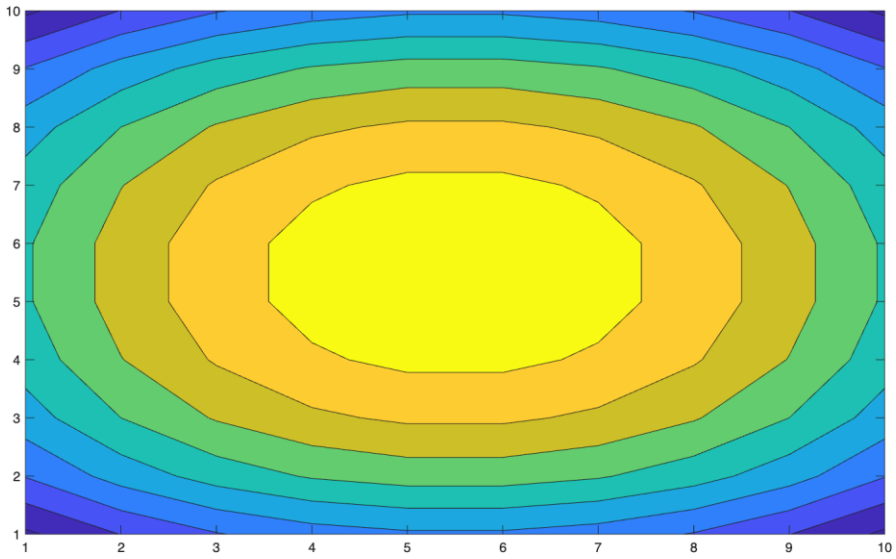


Figure 5: Temperature contours at $h_y = 100\ \frac{W}{m^2 \cdot K}$, $k_x = 1\ \frac{W}{m \cdot K}$, $k_y = 14\ \frac{W}{m \cdot K}$

When comparing figures 5 and 6, we can visualize the effects of altering the conduction coefficients while keeping the convection coefficients constant. Increasing the conduction coefficient in the y direction while decreasing in the x direction causes the temperature to distribute faster y direction. This follows intuition about conduction, the higher the coefficient is the faster heat can move through a material which is why the distribution abrupt in the y direction and gradual in the x direction.

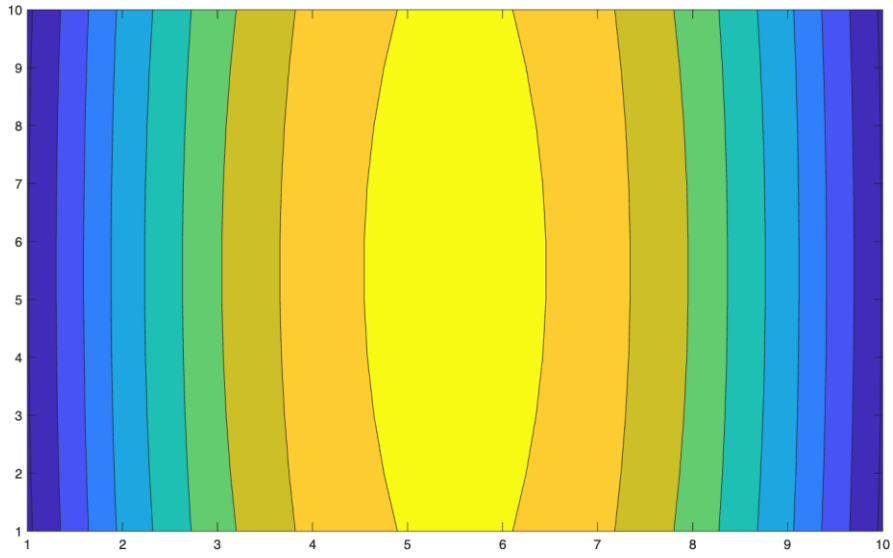


Figure 6: Temperature contour with $h_y = 100\ \frac{W}{m^2 \cdot K}$, $k_x = 1\ \frac{W}{m \cdot K}$, $k_y = 250\ \frac{W}{m \cdot K}$

In closing, a grid independence study was conducted to ensure solution accuracy, using maximum temperature as the benchmark, and it was determined that an 11×11 grid provided sufficiently accurate results while minimizing computational cost. To assess the effect of convection, temperatures were evaluated at five evenly spaced nodes along both the x- and y-centerlines for convective coefficients ranging from h=10 to 100. Results showed that increasing

h consistently reduced temperatures across all nodes, confirming that stronger convection enhances heat removal, with temperature profiles decreasing at nearly parallel rates. Contour plots further illustrated these effects, showing that higher convection coefficients promote greater heat spreading in the x-direction due to increased dissipation at the boundaries. Additionally, varying conduction coefficients revealed that increasing conductivity in the y-direction while decreasing it in the x-direction accelerates heat transfer along y, producing steeper temperature gradients in that direction and more gradual variation in x, consistent with fundamental heat conduction principles.

Conclusion

This study successfully developed code to predict the steady state two-dimensional temperature distribution of electronics packed into a container. These electronics were subjected to internal heat generation and convective cooling. A grid independence study was conducted and confirmed that an 11x11 grid would provide accurate results to minimize computational time. The analysis also demonstrated that increasing the convection coefficient on the y face will decrease internal temperature at all nodes. Using contour plots further confirmed that stronger convective cooling creates more uniform heat distribution in the x direction. The comparison of different thermal conductivity configurations showed that increasing ky and decreasing kx will cause heat transfer greatly to increase in the y direction while it will gradually increase along the x direction. From the analysis and code produced, this study can be used to observe complex conditions in future heat transfer studies.

Appendix

```
function [T_matrix,T_center,T_quad] = nodesolver(gridrows,gridcolumns,hy,kx,ky)
% Finds the temperature gradient across an m by n grid
T_fluid = 25; % degrees celsius
k_x = kx; % W/mK
k_y = ky; %W/mK
q_dot = 250000; % W/m^3
h_x = 100; % W/m^2K
h_y = hy; % W/m^2K (change between 5 values)
total_x = .05; % length of cross section in m
total_y = .2; %height of cross section in m
m = gridrows; % number of rows (changes)
n = gridcolumns; % number of columns (changes)
del_x = total_x/n;
del_y = total_y/m;
T = sym('T',[round(m/2) round(n/2)]); % matrix of t values
%eq = zeros([round(m/2) round(n/2)]); % matrix of equations

%% go through each node and define a heat equation
for r = 1:round(m/2)
   for c = 1:round(n/2)
%% first corner
      if r == 1 && c==1 %|| r == 1 && mod(c, 2) ~= 0 || c==1 && mod(r, 2) ~= 0 || mod(c,2) ~= 0 &&
mod(r,2) ~= 0
         area = (del_x).*(del_y)/4;
                  L_y = del_y/2;
                  L_x = del_x/2;
                  q_left = h_x*L_y*(T_fluid-T(r,c)); % convection
                  q_up = h_y*L_x*(T_fluid-T(r,c)); % convection
                  q_down = k_y*L_x*(T(r+1,c)-T(r,c))/del_y;
                  q_right = k_x*L_y*(T(r,c+1)-T(r,c))/del_x;
         eq = q_left + q_right + q_up + q_down + q_dot*area == 0;
         eq_mat(r,c) = eq;
      %% eq(r,c) = q_left + q_right + q_up + q_down + q_dot*area == 0
%% top side
      elseif r == 1 && c ~= round(n/2) && c~=1
            area = del_x*del_y/2;
            L_y = del_y/2;
            L_x = del_x;
                  q_left = k_x*L_y*(T(r,c-1)-T(r,c))/del_x;
                  q_right = k_x*L_y*(T(r,c+1)-T(r,c))/del_x;
                  q_up = h_y*L_x*(T_fluid-T(r,c)); % convection
                  q_down = k_y*L_x*(T(r+1,c)-T(r,c))/del_y;
         eq = q_left + q_right + q_up + q_down + q_dot*area == 0;
         eq_mat(r,c) = eq;
 % alexs contributions
 %top right corner even
```

```
    elseif r == 1 && c == round(n/2) && mod(n, 2) == 0
            area = del_x * del_y/2;
            eq = h_y * del_x * (T_fluid-T(r,c)) + k_x * del_y/2 * (T(r,c-1) - T(r,c))/del_x + k_y * del_x *
(T(r+1,c) - T(r,c))/del_y + q_dot*area == 0;
        eq_mat(r,c) = eq;
 %top right corner odd
    elseif r == 1 && c == round(n/2) && mod(n, 2) ~= 0
            area = del_x * del_y/4;
            q_left = k_x * del_y/2 *(T(r,c-1)-T(r,c))/del_x;
        q_right = 0;
        q_up = h_y * del_x/2 * (T_fluid-T(r,c));
        q_down = k_y * del_x/2 * (T(r+1,c)-T(r,c))/del_y;
        eq = q_left + q_right + q_up + q_down + q_dot*area == 0;
        eq_mat(r,c) = eq;
%left side
    elseif r~=1 && r ~= round(m/2) && c == 1
            area = del_x.*del_y/2;
        q_left = h_x*del_y*(T_fluid - T(r,c));
        q_right = k_x*del_y*(T(r,c+1) - T(r,c))/del_x;
        q_up = k_y*del_x/2*(T(r-1,c) - T(r,c))/del_y;
        q_down = k_y*del_x/2*(T(r+1,c) - T(r,c))/del_y;
        eq = q_left + q_up + q_right + q_down + q_dot * area == 0;
        eq_mat(r,c) = eq;
%middle
    elseif r ~= 1 && c~= 1 && r ~= round(m/2) && c ~= round(n/2)
                area = del_x * del_y;
        q_down = k_y*del_x*(T(r+1,c) - T(r,c))/del_y;
        q_up = k_y*del_x*(T(r-1,c) - T(r,c))/del_y;
        q_right = k_x*del_y*(T(r,c+1) - T(r,c))/del_x;
        q_left = k_x*del_y*(T(r,c-1) - T(r,c))/del_x;
        eq = q_down + q_up + q_right + q_left + q_dot * area==0;
        eq_mat(r,c) = eq;
%Right side
    elseif r~=1 && r~=round(m/2) && c == round(n/2)
        if mod(n,2) ~= 0 %% even number of columns
            area= del_x * del_y/2;
            q_left = k_x*del_y*(T(r,c-1) - T(r,c))/del_x;
            q_right = 0;
            q_up = k_y*del_x/2*(T(r-1,c) - T(r,c))/del_y;
            q_down = k_y*del_x/2*(T(r+1,c) - T(r,c))/del_y;
                eq = q_left + q_up + q_down + q_right + q_dot * area == 0;
            eq_mat(r,c) = eq;
        else
            area=del_x * del_y;
            q_left = k_x*del_y*(T(r,c-1) - T(r,c))/del_x;
            q_right = 0;
            q_up = k_y*del_x*(T(r-1,c) - T(r,c))/del_y;
            q_down = k_y*del_x*(T(r+1,c) - T(r,c))/del_y;
```

```
                eq = q_left + q_up + q_down + q_right + q_dot * area == 0;
            eq_mat(r,c) = eq;
        end
%bottom left corner
    elseif r == round(m/2) && c == 1
            if mod(m, 2) == 0 %% even rows
            q_left = h_x * del_y*(T_fluid - T(r,c));
            q_right = k_x * del_y*(T(r,c+1)-T(r,c))/del_x;
            q_up = k_y * del_x/2*(T(r-1,c)-T(r,c))/del_y;
            q_down = 0;
                eq =  q_left + q_right + q_up + q_down + q_dot * del_x * del_y / 2 == 0;
            eq_mat(r,c) = eq;
        else %% odd
            q_up = k_y * del_x/2 * (T(r-1,c) - T(r,c))/del_y;
            q_down = 0;
            q_left = h_x * del_y/2*(T_fluid - T(r,c));
            q_right = k_x.*del_y/2*(T(r,c+1)-T(r,c))/del_x;
            eq = q_left + q_right + q_up + q_down + q_dot*del_x*del_y/4 == 0;
            eq_mat(r,c) = eq;
        end
%bottom side
    elseif r == round(m/2) && c~= 1 && c ~= round(n/2)
        if mod(m,2) == 0 %% even number of total rows
            area = del_x * del_y;
                eq = k_y * del_x*(T(r-1,c) - T(r,c))/del_y + k_x * del_y*(T(r,c+1) - T(r,c))/del_x + k_x *
del_y*(T(r,c-1) - T(r,c))/del_x + q_dot * area ==0;
            eq_mat(r,c) = eq;
        else
            area = del_x * del_y/2;
                eq = k_y * del_x*(T(r-1,c) - T(r,c))/del_y + k_x * del_y/2*(T(r,c+1) - T(r,c))/del_x + k_x *
del_y/2*(T(r,c-1) - T(r,c))/del_x + q_dot * area ==0;
            eq_mat(r,c) = eq;
        end
%bottom right corner
    elseif  r == round(m/2) && c == round(n/2)
            if mod(m,2) == 0 & mod(n,2) == 0 %% even rows and columns
                eq =  k_x * del_y *(T(r,c-1)-T(r,c))/del_x + k_y * del_x *(T(r-1,c)-T(r,c))/del_y +
q_dot*del_x*del_y == 0;
            eq_mat(r,c) = eq;
        elseif mod(m,2) == 0 & mod(n,2) == 1 %% even rows, odd columns
            eq = k_x* del_y*(T(r,c-1)-T(r,c))/(del_x) + k_y * del_x/2 *(T(r-1,c)-T(r,c))/(del_y) + q_dot * del_x *
del_y /2 == 0;
            eq_mat(r,c) = eq;
        elseif mod(m,2) == 1 & mod(n,2) == 0 %% odd rows, even columns
                eq = k_x* del_y/2*(T(r,c-1)-T(r,c))/(del_x) + k_y * del_x *(T(r-1,c)-T(r,c))/(del_y) +q_dot *
del_x * del_y /2 == 0;
            eq_mat(r,c) = eq;
        elseif mod(m,2) == 1 & mod(n,2) == 1 %% odd rows and columns
```

```
                    eq = k_x * del_y/2*(T(r,c-1)-T(r,c))/(del_x) + k_y * del_x/2 *(T(r-1,c)-T(r,c))/(del_y) + q_dot *
del_x * del_y /4 == 0;
                eq_mat(r,c) = eq;
            end
        end
    end
end
quad_grid_structure = solve(eq_mat,T);

%% turn structure with fractions to matrix with decimals
for r = 1:round(m/2)
    for c = 1:round(n/2)
        varname = sprintf('T%d_%d', r, c);
        quad_grid_mat(r,c) = double(quad_grid_structure.(varname));
    end
end
T_center = quad_grid_mat(round(m/2),round(n/2));
T_quad = quad_grid_mat;
% concatinate the full grid

if mod(n,2) == 0 % columns even
    top_grid = [quad_grid_mat, fliplr(quad_grid_mat)];
    if mod(m,2) == 0 % rows even
        bottom_grid = flipud(top_grid);
    else %% rows odd
        bottom_grid = flipud(top_grid(1:end-1,:));
    end
else % columns odd
    top_right = quad_grid_mat(:,1:end-1); %% remove middle to prevent double counting
    top_grid = [quad_grid_mat, fliplr(top_right)];
    if mod(m,2) == 0 % rows even
        bottom_grid = flipud(top_grid);
    else % rows odd
        bottom_grid = flipud(top_grid(1:end-1,:));
    end
end
total_grid = [top_grid; bottom_grid];
T_matrix = total_grid;


% grid independance plot
h_y = 10; %% W/m^2K
n = 10; % number of rows(and columns)
grid_number_vec = zeros(1, n);
T_center_vec = zeros(1, n);
for i = 1:n
    j = i + 2;
```

```
    grid_number_vec(i) = j^2;
    [~,T,~] = nodesolver(j,j,h_y,7,14);
    T_center_vec(i)= T;
end
T_center_vec
grid_number_vec;
T_center_vec;
plot(grid_number_vec,T_center_vec,'b*','LineWidth',2)
xlabel('Number of Nodes')
ylabel('Temp [C]')
xticks([9 16 25 36 49 64 81 100 121]);




%x centerline plot
h_y_vec = [10 30 50 70 100]
m = 10; %% case of 100 gridpoints as instructed
n = 10;


% intialize vectors
T_1_vec = [];
T_2_vec = [];
T_3_vec = [];
T_4_vec = [];
T_5_vec = [];
for i = 1:length(h_y_vec)
    [~,~,T] = nodesolver(m,n,h_y_vec(i),7,14) %% temp distribution at each h_y value
    T_1_vec = [T_1_vec, T(round(m/2),1)]; %% temp at specific point at each y value
    T_2_vec = [T_2_vec, T(round(m/2),2)];
    T_3_vec = [T_3_vec, T(round(m/2),3)];
    T_4_vec = [T_4_vec, T(round(m/2),4)];
    T_5_vec = [T_5_vec, T(round(m/2),5)]
end
plot(h_y_vec,T_1_vec,'-*')
hold on
plot(h_y_vec,T_2_vec,'-*')
plot(h_y_vec,T_3_vec,'-*')
plot(h_y_vec,T_4_vec,'-*')
plot(h_y_vec,T_5_vec,'-*')
xlabel('h[W/m^2k')
ylabel('Temp[C]')
legend('Node 1','Node 2','Node 3','Node 4','Node 5')
title('X centerline plot')
```

```
% y centerline plot
h_y_vec = [10 30 50 70 100]
m = 10; %% case of 100 gridpoints as instructed
n = 10;


% intialize vectors
T_1_vec = [];
T_2_vec = [];
T_3_vec = [];
T_4_vec = [];
T_5_vec = [];
for i = 1:length(h_y_vec)
    [~,~,T] = nodesolver(m,n,h_y_vec(i),7,14) %% temp distribution at each h_y value
    T_1_vec = [T_1_vec, T(1,round(n/2))]; %% temp at specific point at each y value
    T_2_vec = [T_2_vec, T(2,round(n/2))];
    T_3_vec = [T_3_vec, T(3,round(n/2))];
    T_4_vec = [T_4_vec, T(4,round(n/2))];
    T_5_vec = [T_5_vec, T(5,round(n/2))];
end
plot(h_y_vec,T_1_vec,'-*')
hold on
plot(h_y_vec,T_2_vec,'-*')
plot(h_y_vec,T_3_vec,'-*')
plot(h_y_vec,T_4_vec,'-*')
plot(h_y_vec,T_5_vec,'-*')
xlabel('h[W/m^2k')
ylabel('Temp[C]')
legend('Node 1','Node 2','Node 3','Node 4','Node 5')
title('Y centerline plot')




% contour plot 1(h_y = 10)
[T_mat,~,~] = nodesolver(10,10,10,7,14)
contourf(T_mat)



% contour plot 2( h_y = 100)
```

```
[T_mat,~,~] = nodesolver(10,10,100,7,14)
contourf(T_mat)


% contour plot 3(h_y = 100, k_x = 1, k_y = 250)
[T_mat,~,~] = nodesolver(10,10,100,1,250)
contourf(T_mat)
```